

Selected TLS-Anvil RFC 5246 Coverage

This note covers the TLS-Anvil items selected on 2026-06-13. I inspected the extracted `results.zip` contents under `/private/tmp/tlsanvil-results-20260613-codex`. The archive did not contain PCAP/PCAPNG files; the concrete evidence below therefore comes from `_containerResult.json`, `*_testsuite.log`, and `*_tlsattacker.log`.

Observed TLS-Anvil Results

TLS-Anvil item	Affected library/version in results	Observed message flow or value
<code>sendAdditionalExtension</code>	GnuTLS 3.7.0 client, MatrixSSL 4.3.0 client, s2n 0.10.24 client	Client was expected to send a fatal alert but continued with <code>ClientKeyExchange</code> , <code>ChangeCipherSpec</code> , and <code>Finished</code> or left the socket open. The extra extension type was not serialized in the JSON logs.
<code>selectUnproposedCompressionMethod</code>	s2n 0.10.24 client	ServerHello selected unoffered compression methods such as <code>DEFLATE</code> and <code>LZS</code> ; client did not produce the expected fatal alert.
<code>acceptsMissingSignature</code>	s2n 0.10.24 client	ServerKeyExchange signature field omitted; client accepted or kept the socket open.
<code>invalidServerKeyExchangeSignature</code>	s2n 0.10.24 client	Invalid SKE signatures included bitmask/bit-position mutations such as <code>SIGNATURE_BITMASK=0,63,69,101,127,255,511</code> and <code>BIT_POSITION=0..7</code> .
<code>acceptsUnproposedNamedGroup</code>	s2n 0.10.24 client	SKE used groups not offered by the client, including Brainpool, X25519/X448, and SECP/SECT families.
<code>clientMustSendCertMsg</code>	MatrixSSL 4.3.0 client	After <code>CertificateRequest</code> , the client did not send <code>Certificate</code> ; failed cases included <code>RECORD_LENGTH=1</code> and groups <code>SECP256R1</code> , <code>SECP384R1</code> , <code>SECP521R1</code> .
<code>sendNotDefinedRecordTypesWithServerHello</code> , <code>sendNotDefinedRecordTypesWithCCSAndFinished</code>	s2n 0.10.24 client; MatrixSSL 4.3.0 client/server; wolfSSL 4.5.0 server	Undefined record content-type byte was not serialized. Expected <code>unexpected_message</code> ; actual behavior was no alert, socket open, decode error, or an unplanned workflow.
<code>acceptAnyRecordVersionNumber</code>	MatrixSSL 4.3.0 server	Test used arbitrary <code>{03,XX}</code> record-layer ClientHello versions; exact <code>XX</code> was not serialized. Server returned fatal <code>illegal_parameter</code> .
<code>versionGreaterThanSupportedByServer</code>	MatrixSSL 4.3.0 server, wolfSSL 4.5.0 server	TLS-Attacker log records ClientHello version <code>03 0F</code> ; MatrixSSL returned fatal <code>protocol_version</code> or <code>decode_error</code> .

TLS-Anvil item	Affected library/version in results	Observed message flow or value
<code>includeUnknownExtension</code> , <code>leaveOutExtensions</code> , <code>offerManyCipherSuites</code>	MatrixSSL 4.3.0 server, wolfSSL debug-x server	Unknown extension type was not serialized. <code>offerManyCipherSuites</code> included unknown/private suite values such as <code>DF 00..DF C5</code> , <code>DF FD</code> , <code>DF FF</code> ; servers often produced decode errors or incomplete workflows.
<code>checkExtensions</code> , <code>serverRandom</code>	MatrixSSL 4.3.0 server, wolfSSL 4.5.0 server	ServerHello field/random failures were reported, but the concrete bad random value was not serialized.
<code>offerManyAlgorithms</code>	LibreSSL 3.2.3 server; MatrixSSL 4.3.0 server; wolfSSL 4.5.0/debug-x server	Many SignatureAndHashAlgorithm values were offered; concrete full list was usually not serialized. Unknown sig/hash cases logged values like <code>03 0F</code> .
<code>includeUnknownSignatureAndHashAlgorithm</code>	MatrixSSL 4.3.0 server, wolfSSL 4.5.0 server	Unknown SignatureAndHashAlgorithm value logged as <code>03 0F</code> ; server often returned decode/protocol errors or did not complete the planned workflow.
<code>signatureIsValid</code>	MatrixSSL 4.3.0 server, wolfSSL 4.5.0/debug-x server	TLS-Anvil expected a valid signed SKE path; affected runs did not execute the expected workflow to completion.

Existing Scenario/RFC Coverage

Item	Existing coverage before this change
<code>sendAdditionalExtension</code>	Covered for model-known TLS 1.2 ServerHello extensions by <code>scen25-scen31</code> in <code>scenario/rfc/5246-core.mau</code> , backed by ServerHello extension RFC labels.
<code>selectUnproposedCompressionMethod</code>	Covered by <code>scen24</code> , which selects <code>zlib-compression</code> even though the client offered only <code>no-compression</code> .
<code>invalidServerKeyExchangeSignature</code>	RFC label existed, but there was no RFC scenario. Added as <code>scen46</code> .
<code>acceptsUnproposedNamedGroup</code>	RFC label existed, but there was no TLS 1.2 SKE scenario. Added as <code>scen51</code> .
<code>clientMustSendCertMsg</code>	Happy-flow client-auth scenarios require the message, but there was no omission scenario. Added as <code>scen47</code> .
<code>versionGreaterThanSupportedByServer</code>	Covered only when interpreted as the ClientHello/ServerHello handshake protocol field, e.g. <code>scen5</code> . Record-layer version compatibility is not covered.
<code>leaveOutExtensions</code>	Covered for model-known mandatory/required extensions, e.g. <code>scen8</code> , <code>scen44</code> , <code>scen45</code> .
<code>offerManyCipherSuites</code>	Existing corpus had many single substitutions, not one large offered vector. Added as <code>scen48</code> .
<code>checkExtensions</code>	Covered for model-known duplicate/forbidden/unrequested extension checks, mainly <code>scen25-scen31</code> .

Item	Existing coverage before this change
<code>offerManyAlgorithms</code>	Existing corpus had one-at-a-time additions, not one large <code>signature_algorithms</code> vector. Added as <code>scen49</code> .
<code>signatureIsValid</code>	TLS 1.3 CertificateVerify signature scenarios existed. Added a TLS 1.2 SKE valid-signature reachability scenario as <code>scen50</code> .

Added RFC 5246 Scenarios

These were added to `maude/scenario/rfc/5246-core.maude`.

New scen	TLS-Anvil item	Behavior deviation / property
<code>scen46</code>	<code>invalidServerKeyExchangeSignature</code>	<code>setM(#signature, mv[noNonce])</code> on <code>buildServerKeyExchangeV2</code> ; client rejects with <code>decrypt-error</code> .
<code>scen47</code>	<code>clientMustSendCertMsg</code>	<code>skip()</code> on <code>buildClientCertificateV2</code> ; server rejects the following <code>ClientKeyExchange</code> with <code>unexpected-message</code> .
<code>scen48</code>	<code>offerManyCipherSuites</code>	Adds a larger cipher-suite vector while preserving <code>TLS-ECDHE-ECDSA-WITH-AES-128-GCM-SHA256</code> ; handshake reaches both closed states.
<code>scen49</code>	<code>offerManyAlgorithms</code>	Replaces <code>signature_algorithms</code> with a larger list including <code>{ecdsa, sha256}</code> ; handshake reaches both closed states.
<code>scen50</code>	<code>signatureIsValid</code>	Baseline valid TLS 1.2 SKE signature path; SKE is processed and the handshake closes normally.
<code>scen51</code>	<code>acceptsUnproposedNamedGroup</code>	After <code>processClientHelloV2</code> , sets server selected supported group to <code>secp384r1</code> ; generated SKE is rejected with <code>illegal-parameter</code> because the client offered <code>secp256r1</code> .

Verification command:

```
printf 'red runScenario(TLS-12, initialCWA, initConf1, behaviorDeviationSpecification46,
scenarioProperty46) .\nred runScenario(TLS-12, initialCWA, initConf1,
behaviorDeviationSpecification47, scenarioProperty47) .\nred runScenario(TLS-12, initialCWA,
initConf1, behaviorDeviationSpecification48, scenarioProperty48) .\nred runScenario(TLS-12,
initialCWA, initConf1, behaviorDeviationSpecification49, scenarioProperty49) .\nred
runScenario(TLS-12, initialCWA, initConf1, behaviorDeviationSpecification50,
scenarioProperty50) .\nred runScenario(TLS-12, initialCWA, initConf1,
behaviorDeviationSpecification51, scenarioProperty51) .\n' \
| ./maude/maude-3.5.1/maude -no-advise requirements/5246-core.maude
```

All six reductions returned `NeList{Scen}`. I also checked representative tester/target generation: client-side TLS-Anvil items with `tester(N2 . SI)` `target(N1 . CI)` generate server-side test actions, and server-side items with `tester(N1 . CI)` `target(N2 . SI)` generate client-side test actions.

Not Immediately Addable

Item	Why not immediate	Minimal model change plan
<code>acceptsMissingSignature</code>	<code>#signature</code> can be corrupted, but <code>remove(#signature)</code> is not implemented. The TLS 1.2 SKE RFC predicate also returns true when the signature field is absent.	Add <code>remove(#signature)</code> semantics for <code>signature(...)</code> ; add an RFC 5246 SKE label requiring a signature for DHE/ECDHE authenticated suites; map the alert to <code>decrypt-error</code> or the RFC-

Item	Why not immediate	Minimal model change plan
		appropriate fatal alert; then add the missing-signature scenario.
<code>sendNotDefinedRecordTypesWith*</code>	<code>ContentType</code> is a closed sort with only <code>handshake</code> , <code>change-cipher-spec</code> , <code>alert</code> , and <code>application-data</code> .	Add an unknown/invalid content-type constructor, include it in all-values and generation mappings, and add record-layer RFC checks that reject undefined types before message-specific parsing.
<code>acceptAnyRecordVersionNumber</code>	<code>#version</code> can mutate the record version, but TLS 1.2 record-version compatibility rules are not checked as RFC properties. Arbitrary <code>{03,XX}</code> versions are not modeled.	Add record-layer version predicates for TLS 1.0/1.1/1.2 compatibility, extend <code>ProtocolVersion</code> or add raw record-version values if SSL3/arbitrary bytes are needed, and wire the checks into receive rules.
<code>includeUnknownExtension</code>	Unknown/GREASE extension IDs are not represented; only known extension constructors can be added or removed.	Add <code>unknown-extension(Nat)</code> or GREASE extension constructors, update duplicate/extension traversal helpers, add client/server ignore-or-reject RFC rules, and expose Java scenario-generation mappings if needed.
<code>serverRandom</code>	Randoms are symbolic <code>Nonce</code> values; byte patterns, timestamp structure, zero/random-quality checks are not modeled.	Add symbolic bad-random constructors such as <code>zeroRandom</code> , <code>gmtUnixTimePattern</code> , or <code>repeatedRandom</code> ; add <code>ServerHello</code> random validation properties if these are to be treated as scenario properties.
<code>includeUnknownSignatureAndHashAlgorithm</code>	Signature algorithms are a closed <code>{AuthenticationAlgorithm, HashAlgorithm}</code> vocabulary; true <code>03 0F</code> or GREASE values cannot be represented.	Add unknown auth/hash or raw signature-scheme constructors, update <code>authC/hashC</code> helpers and selection predicates, and map them in all-values / Java generation.

Classification Summary

Group	Classification
Already covered by existing RFC scenarios	<code>sendAdditionalExtension</code> for known extensions, <code>selectUnproposedCompressionMethod</code> , known <code>leaveOutExtensions</code> , known <code>checkExtensions</code> , handshake-field <code>versionGreaterThanSupportedByServer</code> .
Added now	<code>invalidServerKeyExchangeSignature</code> , <code>acceptsUnproposedNamedGroup</code> , <code>clientMustSendCertMsg</code> , <code>offerManyCipherSuites</code> , <code>offerManyAlgorithms</code> , TLS 1.2 SKE <code>signatureIsValid</code> .
Requires model work	<code>acceptsMissingSignature</code> , undefined record content types, exact record-version compatibility, true unknown extensions, concrete <code>serverRandom</code> checks, true unknown signature/hash algorithms.